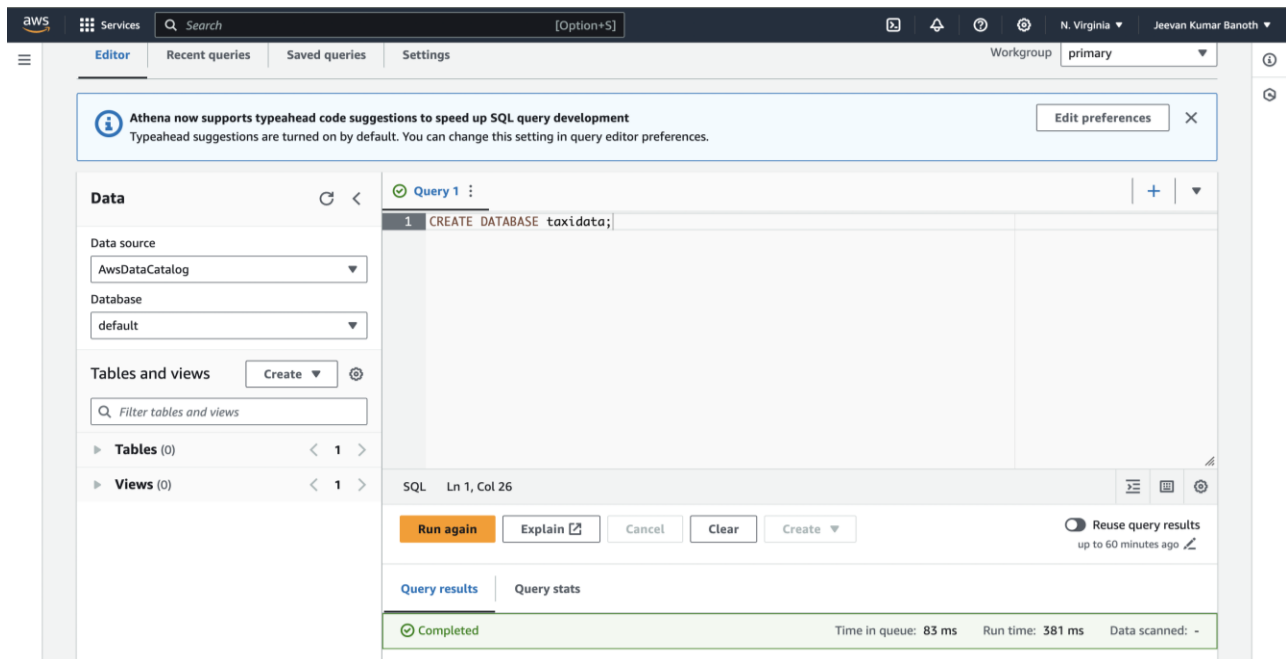


Jeevan Kumar Banoth - 02105145
CIS 602-7103: Special Topics in CIS

Homework – 2:

Step 1: Access the AWS (Amazon Web Services) Management Console: -

- Click "Start Lab" to get the lab session started.
- The session is about to initialize, as indicated by the green circle indicator next to the AWS link.
- Click the AWS link to open the AWS Management Console in a new tab.
- As my AWS student account is not working properly, I will do this by using my own account.
- Find and choose Athena from the AWS Management Console.
- Select Explore the query editor in the Athena console.
- Select the Settings tab by clicking on it.
- Select Manage from the Query result and encryption settings section.
- To view the query result's location, select Browse S3.
- Click Choose after selecting the bucket that was made for you.
- Click Save to save the default configurations.



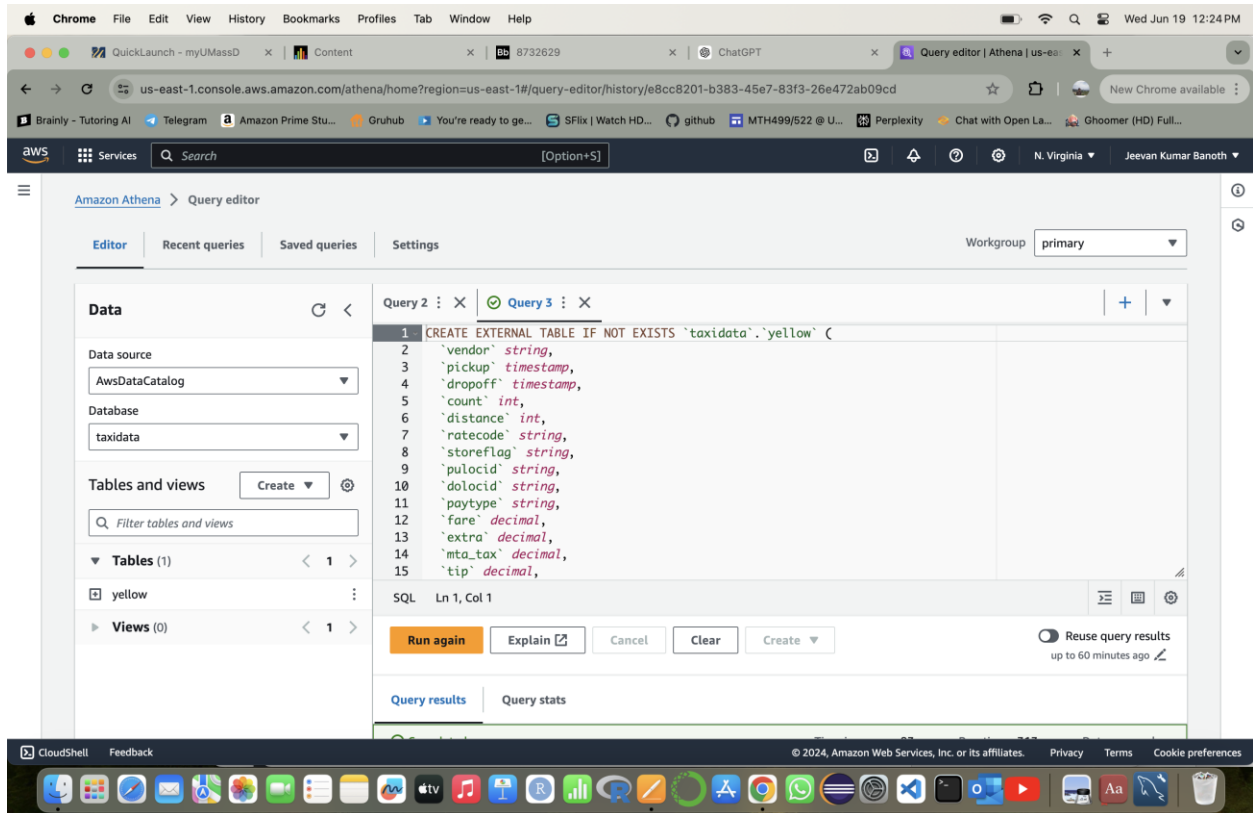
As you can see above, we used the command provided in the lab instructions to build the "taxi data" database in Athena.

In the Athena console, under **Tables and views** on the left, click **Create > S3 bucket data**. Configure the following settings:

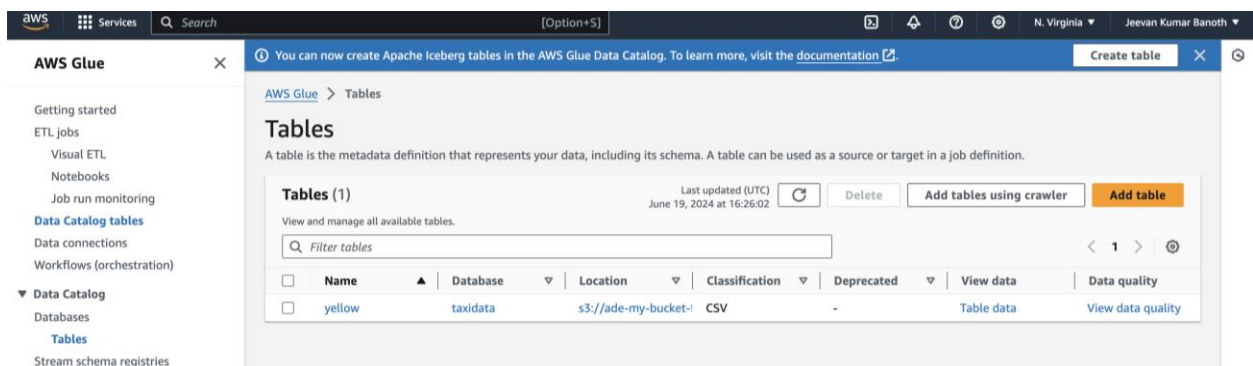
- **Table name:** yellow
- **Description:** Table for taxi data
- **Database configuration:** Select **Choose an existing database** and select taxi data.

- **Encryption:** Keep the default setting (not selected).
- **Data format:** Choose **CSV**.

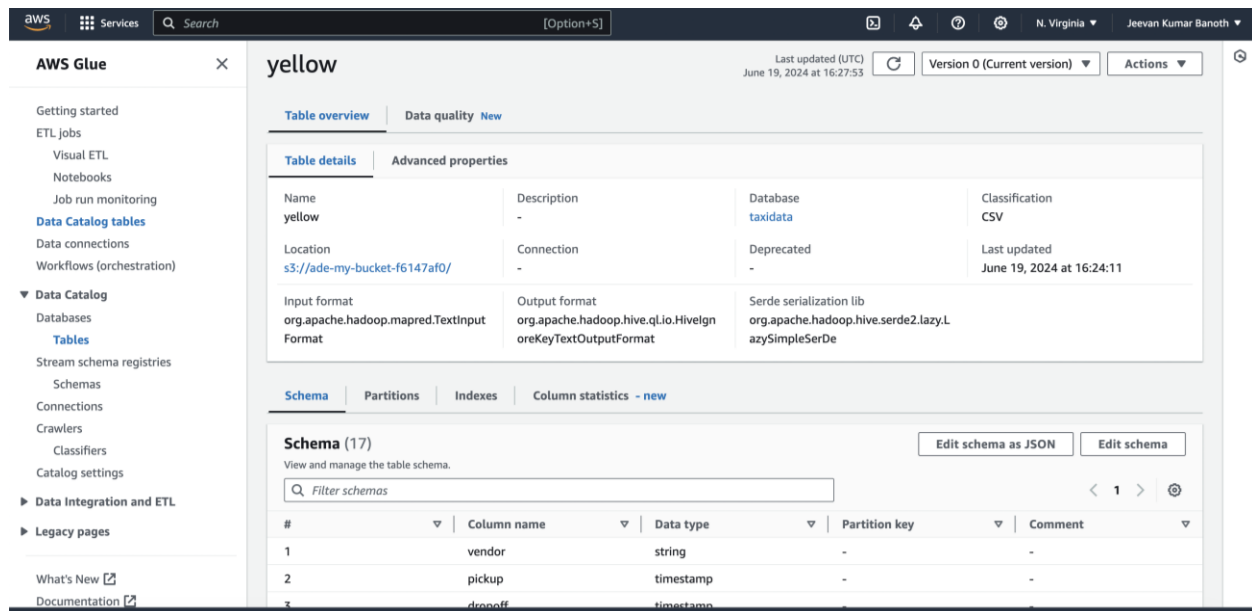
Next, we will be creating a table with the given commands in the manual as below.



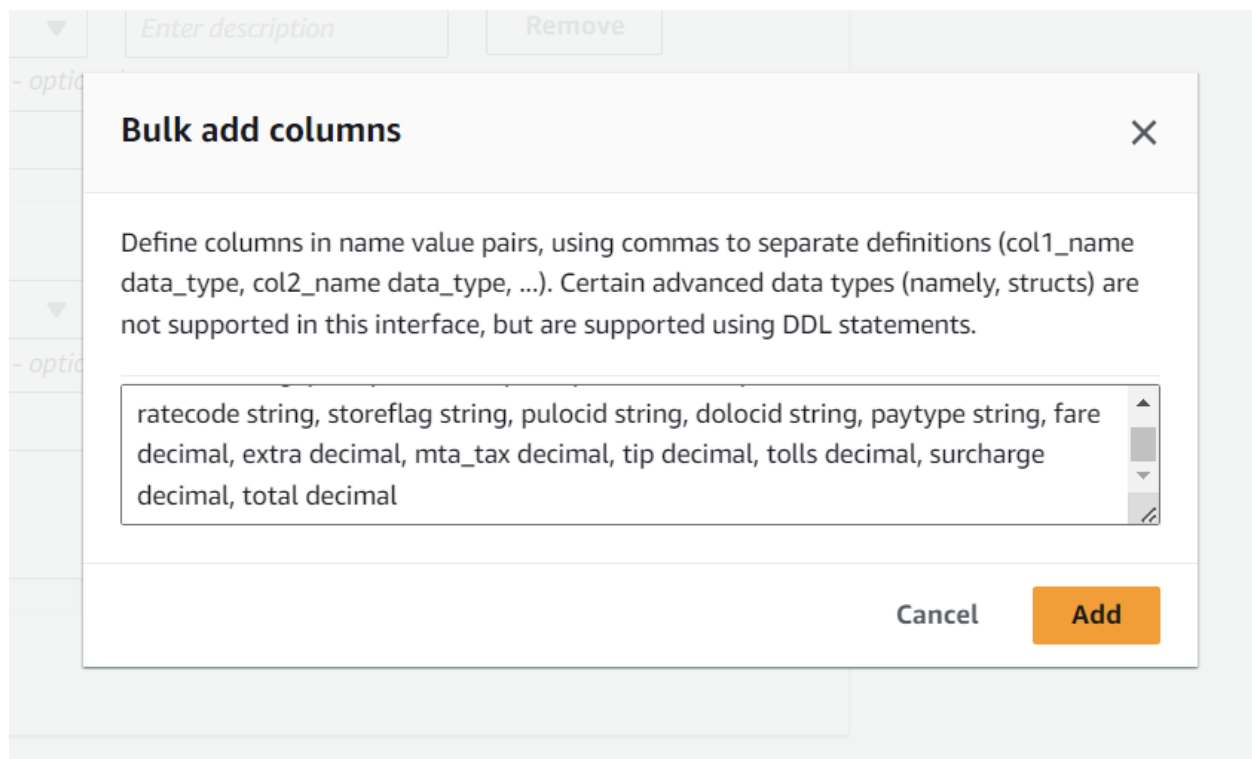
After this, we will be checking in the AWS glue to see if the table has been created or not.



The table named “Yellow” has been created successfully.



Next, by using the bulk add columns, we will add the provided meta data.



As you can see above, we have added all the data which is shown in the lab manual.

Recording the result of the dataset of first 10 dataset values. This is done by using SQL editor as below.

The screenshot shows the AWS Athena console interface. On the left, the 'Data' sidebar is visible, showing the data source 'AwsDataCatalog', database 'taxidata', and a table named 'yellow'. The main area displays the SQL query: `SELECT * FROM "taxidata"."yellow" limit 10;`. Below the query, there are buttons for 'Run again', 'Explain', 'Cancel', 'Clear', and 'Create'. The status bar at the bottom indicates the query is 'Completed' with a 'Time in queue: 104 ms', 'Run time: 896 ms', and 'Data scanned: 0.91 KB'.

The screenshot shows the 'Query results' tab in the AWS Athena console. The status bar at the top indicates the query is 'Completed' with a 'Time in queue: 104 ms', 'Run time: 896 ms', and 'Data scanned: 0.91 KB'. Below the status bar, there are buttons for 'Copy' and 'Download results'. The results are displayed in a table with the following columns: #, vendor, pickup, dropoff, count, distance, ratecode, storeflag, and pulocid. The first three rows show 'FAILED: Execution Error', and the remaining rows show taxi trip data.

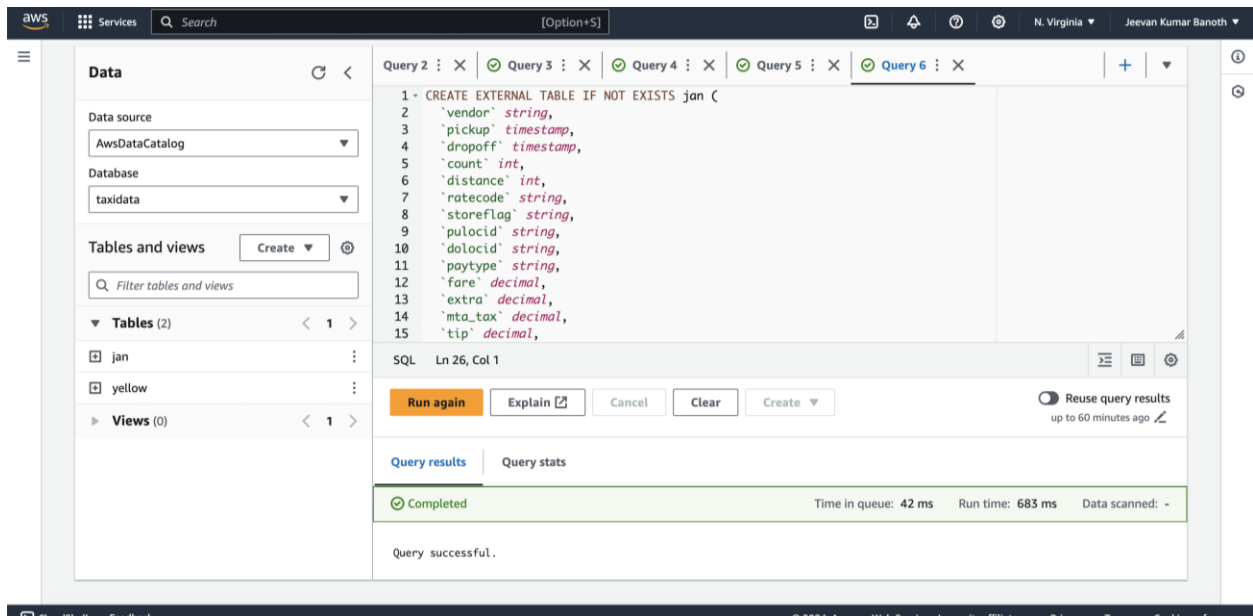
#	vendor	pickup	dropoff	count	distance	ratecode	storeflag	pulocid
1	FAILED: Execution Error							
2	FAILED: Execution Error							
3	FAILED: Execution Error							
4	CustomerID					City	State	Phone
9	CustomerID					City	State	Phone
8	4					Anytown	NY	914-55
7	3					Nowhere	NY	914-55
6	2					Anywhere	WA	360-55
5	1					Any Town	MD	301-55
10	1					Any Town	MD	301-55

Result above shows that, query has been runs successfully in 896ms (about 1 second) as runtime. And the below values show the result of it.

Task 2: Focusing on optimizing Athena queries using bucketizing: -

Data querying can be optimized in two ways: first by employing "buckets," and then by "partitioning." The taxi data is grouped according to the months.

As a result, we are building a table called "jan" that will hold the taxi information for January 2017.



We can now query both the main table and the bucket table to see the power of bucketing.

The following is a query to obtain details about taxis in January, but from the "yellow" source table.

The screenshot shows the AWS Glue console interface. On the left, the 'Data' sidebar is visible with 'AwsDataCatalog' as the data source and 'taxidata' as the database. Under 'Tables and views', the 'yellow' table is selected. The main editor displays a SQL query (Query 7) that counts trips and sums fares for the 'yellow' table, filtered by pickup time between January 1, 2017, and February 1, 2017. The query is executed, and the status bar indicates 'Completed' with a run time of 555 ms and 0.91 KB of data scanned. The 'Results' section shows 0 results.

```
1 SELECT count(count) AS "Number of trips",
2       sum(total) AS "Total fares",
3       pickup AS "Trip date"
4 FROM yellow
5 WHERE pickup BETWEEN TIMESTAMP '2017-01-01 00:00:00'
6       AND TIMESTAMP '2017-02-01 00:00:01'
7 GROUP BY pickup;
```

SQL Ln 8, Col 1

Run again Explain Cancel Clear Create

Query results Query stats

Completed Time in queue: 108 ms Run time: 555 ms Data scanned: 0.91 KB

Results (0) Copy Download results

This shows that it scanned 0.91KB of data and took 555ms (about half second) to retrieve the data. In contrast, the "Jan" bucketing table is used for querying data.

The screenshot shows the AWS Glue console interface. On the left, the 'Data' sidebar is visible with 'AwsDataCatalog' as the data source and 'taxidata' as the database. Under 'Tables and views', the 'jan' table is selected. The main editor displays a SQL query (Query 7) that counts trips and sums fares for the 'jan' table, filtered by pickup time between January 1, 2017, and February 1, 2017. The query is executed, and the status bar indicates 'Completed' with a run time of 5.962 sec and 815.30 MB of data scanned. The 'Results' section shows 2,387,451 results.

```
1 SELECT count(count) AS "Number of trips",
2       sum(total) AS "Total fares",
3       pickup AS "Trip date"
4 FROM jan
5 GROUP BY pickup;
```

SQL Ln 6, Col 1

Run again Explain Cancel Clear Create

Query results Query stats

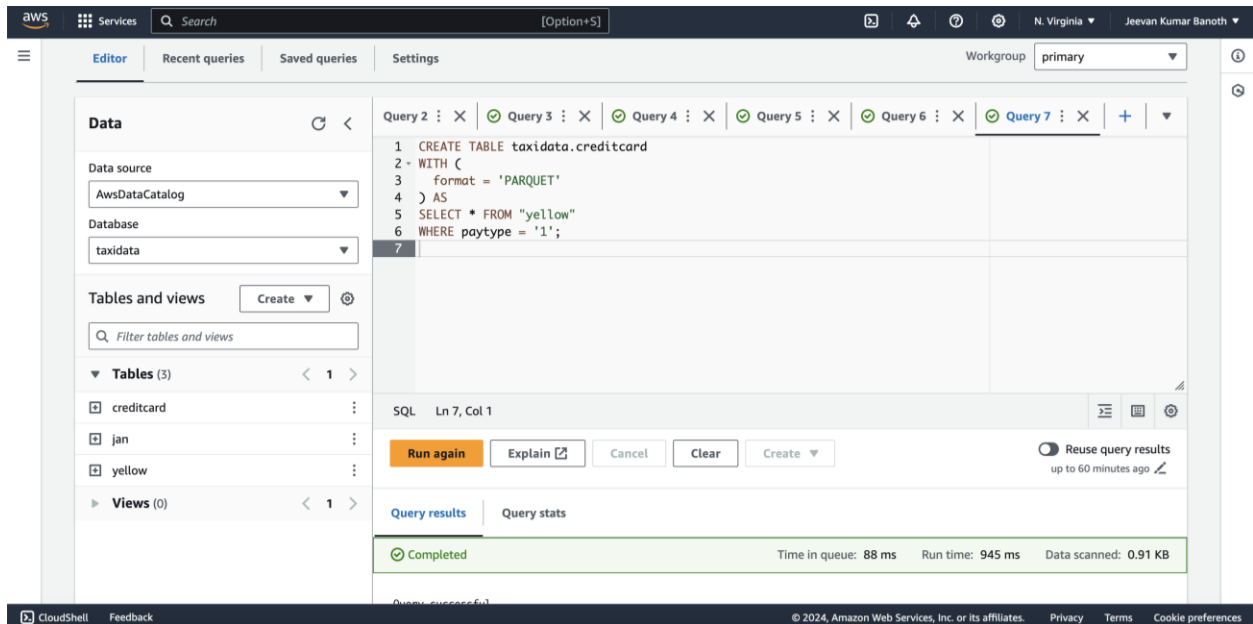
Completed Time in queue: 64 ms Run time: 5.962 sec Data scanned: 815.30 MB

Results (2,387,451) Copy Download results

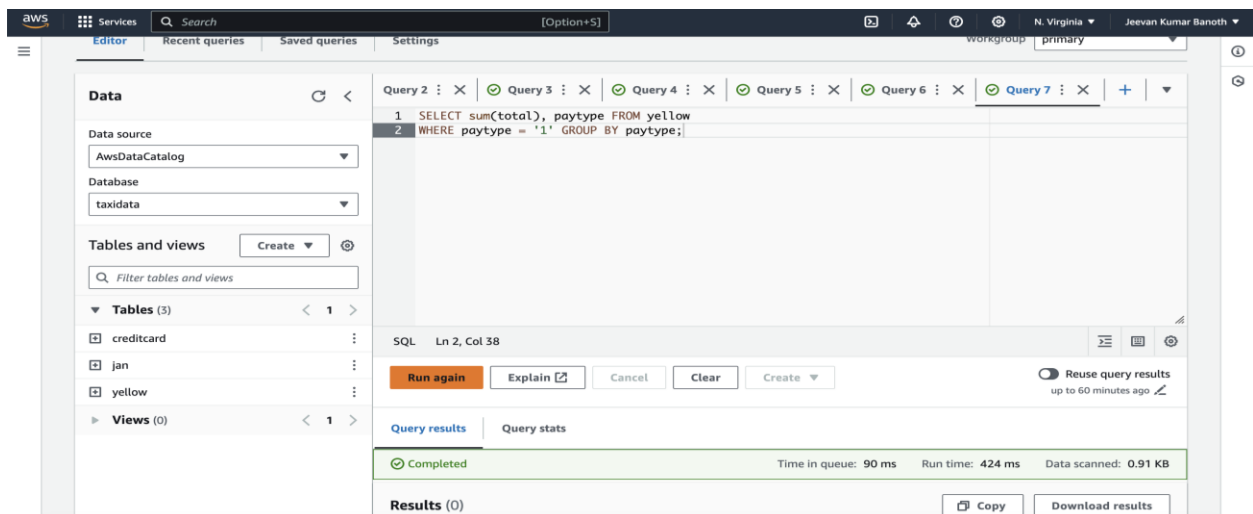
This shows that it scanned 815.30 MB of data and took 5.962 sec to retrieve the data. So this shows the power of using bucket in the table.

Task 3: Focusing on optimizing Athena queries using partitions:

- Using "partitions" is the next data optimization technique.
- When data columns have a high cardinality, such months or departments, bucketing is applied.
- Low cardinality columns can also have partitions constructed on them.
- Here, we're building a table called "Creditcard" from the "yellow" table, setting the paytype to 1, and saving it in a Parquet file.



From this, we can see that this query was completed in 945ms (about 1 second) and data scanned was 0.91 KB.



Querying partitioned data improved performance and significantly reduced the amount of data scanned, demonstrating the effectiveness of partitioning for low cardinality fields.

Task 4: Using Athena Views: -

Comparable to SQL views, these allow us to use views to query data across several tables.

Consequently, we start by creating a view called "cctrips" with paytype of 1.

The screenshot shows the AWS Athena console interface. On the left, the 'Data' sidebar is visible with 'Data source' set to 'AwsDataCatalog' and 'Database' set to 'taxidata'. Under 'Tables and views', there are three tables listed: 'creditcard', 'jan', and 'yellow'. There is one view listed: 'cctrips'. The main area displays the SQL query for creating the 'cctrips' view:

```
1 CREATE VIEW cctrips AS
2 SELECT "sum"("fare") "CreditCardFares"
3 FROM yellow
4 WHERE ("paytype"='1');
```

Below the query editor, the 'Query results' tab is active, showing a green bar indicating the query is 'Completed'. The query status shows 'Time in queue: 94 ms', 'Run time: 732 ms', and 'Data scanned: -'. The message 'Query successful.' is displayed at the bottom.

Now we replace cctrips with cashtrips:

The screenshot shows the AWS Athena console interface after replacing the 'cctrips' view with 'cashtrips'. The 'Data' sidebar on the left now shows two views: 'cashtrips' and 'cctrips'. The main area displays the SQL query for creating the 'cashtrips' view:

```
1 CREATE VIEW cashtrips AS
2 SELECT "sum"("fare") "CashFares"
3 FROM yellow
4 WHERE ("paytype"='2');
```

Below the query editor, the 'Query results' tab is active, showing a green bar indicating the query is 'Completed'. The query status shows 'Time in queue: 81 ms', 'Run time: 600 ms', and 'Data scanned: -'. The message 'Query successful.' is displayed at the bottom.

Then we will be creating the view which is comparepay:

Data

Data source

AwsDataCatalog

Database

taxidata

Tables and views

Create

Filter tables and views

Tables (3)

creditcard

jan

yellow

Views (3)

cashtrips

cctrips

comparepay

Query 1

Query 2

Query 3

Query 4

Query 5

```

1 CREATE VIEW comparepay AS
2 WITH
3   cc AS
4   (SELECT sum(fare) AS cctotal,
5        vendor
6   FROM yellow
7   WHERE paytype = '1'
8   GROUP BY paytype, vendor),
9   cs AS
10  (SELECT sum(fare) AS cashtotal,
11       vendor, paytype
12  FROM yellow
13  WHERE paytype = '2'
14  GROUP BY paytype, vendor)
15

```

SQL

Ln 20, Col 1

Run again

Explain

Cancel

Clear

Create

Reuse query results

up to 60 minutes ago

Query results

Query stats

Completed

Time in queue: 54 ms

Run time: 618 ms

Data scanned: -

Query successful.

To view the comparepay we will be using below query for 10 values:

Query 1

Query 2

Query 3

Query 4

Query 5

Query 6

```

1 SELECT * FROM "taxidata"."comparepay" limit 10;

```

SQL

Ln 1, Col 1

Run again

Explain

Cancel

Clear

Create

Reuse query results

up to 60 minutes ago

Query results

Query stats

Completed

Time in queue: 150 ms

Run time: 8.347 sec

Data scanned: 18.64 GB

Results (2)

Copy

Download results

From this, we will get the data from the different tables we have created to analyze them.

Conclusion:

- familiarized myself with the fundamentals of SQL and the Athena interface.
- showed how bucketizing high-cardinality data can improve query performance and cut expenses.
- emphasized how splitting low-cardinality data can improve queries and cut expenses.
- shown how views can help with effective data analysis, make difficult queries simpler, and boost efficiency by reusing results.
- To combine data from the two views and compare revenues by vendor, a composite view was created.
- significant gains in query performance were observed, and the amount of data scanned for partitioned data was decreased.
- compared how well searches performed on partitioned versus non-partitioned data.
- investigated a number of query optimization techniques for Athena, such as bucketization, partitioning, and view utilization.
- shown how Athena's query patterns and data storage may be optimized to greatly improve performance and cut expenses.
- Developed practical skills in running and optimizing queries in Athena, beneficial for efficient data analysis and cost management.

-----X-----